

BNPP Test technique — Rapport complet

Candidat : Titouan

Contexte : Test technique Data Science chez **BNP Paribas** via **MERITIS**

Sujet : Implémentation d'une boucle d'entraînement **PyTorch** pour affiner un **MoE** pré-entraîné (**IBM Granite 3.1 – 1.3B / a400m-instruct**) sur **GSM8K** avec **GRPO + LoRA**, déploiement/inférence via **vLLM**, contrainte ≤ 16 Go VRAM et ≤ 4 h d'entraînement.

1) Résumé exécutif

L'objectif est de démontrer un pipeline **RLHF-like** minimal mais robuste pour du raisonnement mathématique: préparation des prompts GSM8K, adaptation **PEFT/LoRA** ciblée sur les projections d'attention, **gel des experts** MoE et **apprentissage du routeur**, entraînement **GRPO** avec **récompense numérique** simple et **pénalisation KL** via une **politique de référence** gelée, le tout **sous 16 Go VRAM** (Kaggle **T4**).

Cheminement compute (CPU local → GPU Kaggle) :

1. **Smoke test CPU** sur mon **ordinateur personnel** pour valider les dépendances, le chargement du tokenizer/modèle et une génération minimale sans CUDA (script `test_granite_cpu.py`).
2. **Passage sur Kaggle** pour disposer d'un **GPU T4 (16 Go)** — parfois 2×T4 sont visibles, mais j'ai **forcé 1 seul GPU** via `CUDA_VISIBLE_DEVICES=0` pour respecter la contrainte. J'y ai **désactivé bf16**, **activé fp16** et **coupé vLLM** pendant l'entraînement pour tenir le budget mémoire/temps.

Choix clés :

- **Modèle :** *ibm-granite/granite-3.1-1b-a400m-instruct* (archi **MoE** compacte, adaptée aux contraintes GPU).
- **Param-Efficient FT : LoRA** sur `q_proj/k_proj/v_proj/o_proj` uniquement → $< 1\%$ des paramètres entraînaibles.
- **MoE policy : experts gelés, routeur entraîné** pour optimiser la sélection d'experts.
- **RL : GRPO** (grouped PPO) + **KL** vs politique gelée → stabilité et régularisation sous faible budget.
- **Format sortie :** réponses numériques `` , anti-bavardage (bad words) et *fallback* de génération.
- **Reproductibilité :** seed, journalisation **TensorBoard**, **checkpointing**.

Contraintes respectées : 1×T4 (16 Go), **AMP fp16** (bf16 off), batch effectif via **accumulation**, longueur de génération **courte** (16–64), ≤ 4 h sur échantillon GSM8K réduit.

2) Architecture globale du repo

```
BNPP-test-technique/
├─ config.py           # Dataclass de configuration (dtype, LoRA, GRPO, I/O)
├─ data_loader.py      # Chargement/formatage GSM8K, extraction des réponses
```

— model_loader.py	# Chargement Granite + LoRA + gel/routeur
— grpo_trainer.py	# Entraîneur GRPO (récompense, KL, PPO-like, gen fallback)
— train.py	# Script principal (args, logs TB, meta, boucle train)
— test_granite_cpu.py	# Sanity check d'inférence CPU
— requirements.txt	# Dépendances (Transformers, TRL/PEFT, etc.)
— README.md	# Guide d'utilisation rapide

Rôles :

- `` : centralise **dtype** (fp16/bf16), **vLLM on/off**, hyperparamètres (**lr**, **accumulation**, **warmup**, etc.).
- ` : charge **\*\*GSM8K/main\*\***, crée des **\*\*prompts stricts\*\*** et des **\*\*réponses gold\*\*** ; limite les tokens d'entrée ; expose un `DataLoader` RL-friendly.
- ` : télécharge **\*\*Granite 3.1 1B (MoE)\*\***, **\*\*gèle\*\*** tous les poids, **\*\*dé-gèle\*\*** le **\*\*routeur\*\*** ( `block_sparse_moe.router.layer.weight`) et **injecte LoRA** sur les projets d'attention.
- ` : entraîneur **\*\*GRPO\*\*** (optim **\*\*AdamW\*\***, **\*\*scheduler linéaire\*\***, **\*\*KL\*\*** vs modèle gelé, **\*\*entropie\*\***), fonction de **\*\*récompense numérique\*\*** (bonus si format `####`), **bad_words_ids** pour éviter « Q:/A: », **fallback** de génération en cas d'échec, **sauvegardes** périodiques.
- `` : parse les arguments CLI, initialise **TensorBoard**, **run_meta.json**, charge modèle+données, instancie le **trainer** et lance l'entraînement.

3) Choix d'architecture & justifications

3.1 Modèle : IBM Granite 3.1 – 1.3B MoE

- **Pourquoi** : compromis **qualité/latence/mémoire** très favorable en 16 Go ; MoE = spécialisations d'experts sans accroître tous les paramètres actifs.
- **Conséquence** : privilégier l'**apprentissage du routeur** pour mieux router vers les bons experts sur la distribution GSM8K.

3.2 Adaptation param-efficient : LoRA ciblée

- **Cibles** : `q_proj, k_proj, v_proj, o_proj` → impact direct sur **attention** (raisonnement, extraction chiffres).
- **Avantage** : < 1 % des params entraînaibles → **VRAM** faible, **vitesse** accrue, **risque de sur-apprentissage** moindre.

3.3 Politique MoE

- **Experts gelés** : protège les compétences de base, évite OOM et instabilité sous petit batch.
- **Routeur appris** : ajuste le **pattern d'activation des experts** vers les tâches arithmétiques GSM8K.

3.4 RL : GRPO + KL

- **GRPO** : variante **PPO** orientée batch/groupe qui **stabilise** l'update politique en **faible compute**.
- **KL vs ref** : un **clone gelé** du modèle sert de **politique de référence** (anti-dérive).
- **Entropie** : bonus modéré pour maintenir un minimum d'exploration.

3.5 Génération contrôlée

- **Format sortie** : forcer ou favoriser `#### <nombre>` pour **auto-évaluation** et **récompense**.
- **Bad words** : bloque « Q: », « A: », etc., pour **réduire le verbiage**.
- **Fallback** : si `generate` échoue, **voie déterministe** courte qui **reformate** la meilleure hypothèse numérique trouvée.

3.6 Contraintes mémoire/temps

- **fp16** (bf16 **off** sur T4), **batch=1** + **accumulation=8**, **max_new_tokens** réduit (16–64), **séquences tronquées**.
- **Checkpoints** espacés, **TensorBoard** pour suivre loss/récompense.
- $\leq 4\text{ h}$: on **échantillonne GSM8K** (64–256 ex.) pour montrer l'**intégration** plus que la SOTA.

4) Détails d'implémentation

4.1 Données — `data_loader.py`

- Chargement **GSM8K/main**.
- Construction de **prompts** type Q/A avec exigence de sortie **numérique** `####` (moins de tokens d'achèvement pour rester frugal).
- **Extraction numérique** robuste (regex) côté reward/eval.
- **Max tokens d'entrée** configurable (par défaut ~ 768) pour maîtriser la **VRAM**.

4.2 Modèle — `model_loader.py`

- **Freeze** des poids, **unfreeze** du **routeur MoE**.
- **LoRA** injectée sur les **projections d'attention** ($r=8$, $\alpha=16$, dropout=0.05).
- Affichage des **paramètres entraînaibles** + stats (trainable vs total) pour traçabilité.

4.3 Entraîneur — `grpo_trainer.py`

- **Optim** : **AdamW** ; **scheduler linéaire** avec warmup.
- **Référence KL** : deep-copy **gelée** du modèle.
- **Récompense** :
 - Parse **le dernier nombre** généré ;
 - **Bonus** si le format `####` est respecté ;
 - Barème par **exact match**, **tolérance absolue** et ****erreur relative****.
- **Avantages** : normalisation des récompenses (stabilité PPO-like).
- **Génération** : déterministe par défaut (température basse), **bad_words_ids**, **fallback** si besoin.
- **Logs** : impression step-wise (loss, reward_mean, entropy), **checkpoints** périodiques, **save final** dans `outputs/grpo-granite`.

4.4 Script principal — `train.py`

- **Arguments** : `--samples`, `--max-steps`, `--new-tokens`, `--temperature`, `--output_dir`, `--log_dir`, `--seed`.
- **TensorBoard** : logs dans `outputs/logs` ; `run_meta.json` pour la traçabilité (seed, samples, steps, new_tokens).

4.5 Sanity check — `test_granite_cpu.py`

- Test de **charge modèle/tokenizer** et **génération** CPU, utile pour **debug** hors GPU.

5) Pipeline d'exécution (CPU perso → Kaggle T4, 16 Go)

5.1 Étape 1 — Smoke test CPU local

- **Objectifs** : valider le chargement du modèle Granite, la tokenization, une génération courte, et vérifier les versions (`transformers`, `trl`, `peft`, `datasets`).
- **Bénéfice** : diagnostic rapide sans dépendre de CUDA/cuDNN (élimine les erreurs type *factory already registered* ou conflits drivers).

5.2 Étape 2 — Kaggle (1× T4 16 Go)

- **Configuration** :
- `CUDA_VISIBLE_DEVICES=0` pour n'utiliser **qu'un seul** T4.
- **bf16 désactivé**, **fp16 activé** via patch sur `config.py`.
- **vLLM désactivé** pendant le training (réactivable pour la démo d'inférence).
- Seed, suppression de logs verbeux, `CUDA_LAUNCH_BLOCKING=1` pour le debug runtime.
- **Raison** : respecter la contrainte ≤ 16 Go VRAM et ≤ 4 h, tout en gardant une perf décente.

5.3 Lancement (exemple Kaggle)

```
python train.py
--samples 64
--new-tokens 16
--max-steps 120
--temperature 0.7
--output_dir outputs/grpo-granite
--log_dir outputs/logs
--seed 42
```

Suivi : `tensorboard --logdir outputs/logs`

Artefacts : `outputs/grpo-granite/` (+ checkpoints intermédiaires).

6) Bonnes pratiques respectées

- **Gestion VRAM** : fp16, batch=1, accumulation=8, prompts courts, new_tokens limités, checkpoints espacés.

- **Stabilité RL** : KL vs ref, **clip PPO**, **grad clip**, **warmup**, **entropy_coef** faible.
- **Qualité sorties** : format strict, *bad words*, **fallback** pour ne jamais bloquer une itération.
- **Observabilité** : TensorBoard, logs step-wise, `run_meta.json`.
- **Reproductibilité** : seed, versions verrouillées, env vars CUDA/JAX/XLA neutralisées.

7) Difficultés rencontrées & solutions

1. VRAM/OOM sur T4

Problème : séquences et achèvements trop longs → OOM.

Solution : réduire `max_input_tokens` (≈ 768), `max_new_tokens` (16-32), **batch=1** + **accumulation**, fp16, éviter le sampling coûteux.

2. Instabilités de génération (verbiage, formats variés)

Problème : sorties non numériques ou hors format.

Solution : **bad_words_ids**, **bonus de format** dans la **récompense**, **fallback** qui **re-encode** `#### <nombre>` si nécessaire.

3. Pénalisation KL numérique (NaN/Inf)

Problème : logprobs `NaN` / `Inf` lors de l'échantillonnage.

Solution : **clamp**/`nan_to_num` des logits/logprobs, température basse, génération **greedy** par défaut dans l'entraîneur.

4. Référence gelée volumineuse

Problème : surcharge mémoire due au **clone** de référence.

Solution : **eval()** + **no_grad()**, réutilisation CPU/GPU prudente, forward minimal (gather sur le **dernier token** seulement).

5. Compatibilité bf16/T4

Problème : T4 ne supporte pas **bf16**.

Solution : **bf16 off**, **fp16 on** (patch auto du `config.py`).

8) Résultats & métriques (runs courts de démonstration)

- **Objectif principal** : prouver la **solidité** de la boucle GRPO sous contraintes, plus que la performance SOTA.
- **Suivi** (TensorBoard) : `loss/total`, `reward/mean`, `entropy`, courbes stables après warmup.
- **Logs** : affichage `[Step k] loss=... reward_mean=... entropy=...` à fréquence régulière, checkpoints toutes ~ 50 itérations.

Remarque : l'accuracy brute sur GSM8K **complet** n'est pas l'objectif de ce run **échantillonné** (temps limité). Une évaluation plus stricte est listée en «Améliorations futures».

9) Ce qu'il reste à faire (roadmap courte)

1. **Évaluation rigoureuse**
2. Mettre en place un **split validation** et un **script d'évaluation** (exact match, tolérance, conformité format), + tableau d'erreurs fréquentes.
3. **Récompense plus riche**
4. Détecter **incohérences** (nombres absurdes), **pénalités de longueur**, **bonus chaîne de calcul** quand présent.
5. **Intégration vLLM**
6. **Serveur vLLM** pour l'inférence rapide **post-training** (quantization optionnelle 8/4 bits), tests de **throughput/latence**.
7. **Optimisations mémoire/perf**
8. **Gradient checkpointing**, **Paged Optimizers**, **offload CPU** (selon infra), **NF4** (bitsandbytes) si nécessaire.
9. **Data curriculum**
10. Commencer par **questions simples**, augmenter la difficulté (preuves courtes → plus longues) ; éventuellement mélanger **GSM8K** & **math-word-problems** proches.
11. **Packaging/CI**
12. Script **Docker/Makefile**, tests unitaires (récompense, parsing), **lint/format** (`pre-commit`).

10) Risques & points de vigilance

- **Reward hacking** : le modèle pourrait « jouer » le barème → diversifier la récompense (consistance, étapes, exactitude).
- **Catastrophic forgetting** : même gelé, un routeur mal entraîné peut **dérouter** ; surveiller KL, prévoir **early stop**.
- **Généralisation** : runs courts → **variance élevée** ; activer éval périodique, **multi-seed**.
- **Conformité & sécurité** (banque) : traçabilité complète des artefacts et dépendances ; éviter tout **leak** de données (ici dataset public).

11) Mode opératoire — Installation rapide

```
pip install -r requirements.txt
# Test CPU rapide\python test_granite_cpu.py
# Entraînement court (démon)
```

```
python train.py --samples 64 --new-tokens 16 --max-steps 120 --temperature 0.7
```

TensorBoard : `tensorboard --logdir outputs/logs`

Modèle finetuné : `outputs/grpo-granite/`

12) Annexes

A. Environnement Kaggle (extraits)

- **Env :** `CUDA_VISIBLE_DEVICES=0`, `TF_CPP_MIN_LOG_LEVEL=3`, `JAX_PLATFORMS=cpu`, `XLA_PYTHON_CLIENT_PREALLOCATE=false`, `CUDA_LAUNCH_BLOCKING=1`, `CUBLAS_WORKSPACE_CONFIG=:4096:8`.
- **Patch config :** `bf16` → `off`, `fp16` → `on`, `vLLM` → `off` (entraînement).
- **Seed :** 42, `cudnn.benchmark=True`.
- **Repo :** clone branch `kaggle-refactor`, sauvegarde `run_meta.json`.

B. Paramètres LoRA par défaut

- `r=8`, `alpha=16`, `dropout=0.05`, cibles : `q_proj/k_proj/v_proj/o_proj`.

C. Hyperparamètres GRPO (par défaut raisonnable T4)

- `lr=5e-5`, `weight_decay=0.01`, `warmup_steps=50`, `clip_range=0.2`, `kl_coef=0.05`, `entropy_coef=0.001`, `max_grad_norm=1.0`.
-

13) Conclusion

Ce livrable met en place **une base propre, traçable et frugale** pour du **RL sur MoE** en contexte **contraint**. Les choix (LoRA ciblée, routeur entraîné, GRPO+KL, format numérique strict, contrôles mémoire) sont **cohérents** avec les exigences BNPP et le **timebox**. La **prochaine itération** portera sur l'**évaluation formelle**, l'**intégration vLLM** en inference-server, et des **optimisations** mémoire/latence.